Feature-Level Interpretability and Causal Interventions in the Tiny Recursive Model on ARC-AGI-1

Shangyi Du, Shangyu Gong, Yong-Hyun Park, Arthur Pogosian

Abstract

We present an interpretability pipeline for a Tiny Recursive Model (TRM) with Adaptive Computation Time on ARC-AGI-1. We log stepwise latent trajectories during evaluation and train a Top-K sparse autoencoder on the lower-level state to obtain sparse features. Features are ranked by activation statistics, and we apply targeted interventions during recurrence (subtractive and reconstruction-based ablations, including progressive schedules) to measure changes in accuracy and error counts. This workflow connects internal features to puzzle-solving behavior and supports causal analysis.

1 Introduction

ARC-AGI-1 puzzles demand compositional reasoning over discrete color grids (Chollet, 2019). Understanding how neural solvers arrive at correct solutions requires interpretability tools that go beyond accuracy metrics.

Our model is a Tiny Recursive Model (TRM) that iteratively refines an internal token state and uses an ACT-style halting mechanism to decide when to stop (Jolicoeur-Martineau, 2025; Graves, 2016). This architecture naturally exposes intermediate signals across "thinking steps," enabling trajectory-level logging. To interpret internal computation, we train a Top-K sparse autoencoder (SAE) on TRM activations, following recent work using SAEs for mechanistic interpretability and behavior attribution (Cunningham et al., 2023; Makhzani and Frey, 2013; Gao, 2025). Our contribution is a project-level methodology: trajectory logging, SAE-based feature analysis, and causal interventions in a recurrent/ACT ARC solver.

2 Related Work

ARC and neural solvers. ARC was introduced by Chollet (2019) as a benchmark for abstraction and reasoning over grids. Many approaches rely on program synthesis, DSL search, or strong priors; neural solvers must learn representations for colors, shapes, and transformations under few-shot supervision.

Recursive reasoning models for puzzles. Hierarchical Reasoning Models (HRM) (Wang et al., 2025) and subsequent simplifications such as TRM (Jolicoeur-Martineau, 2025) demonstrate strong puzzle performance with small networks by using recurrence, deep supervision, and iterative refinement. TRM's iterative nature makes it a good target for trajectory-based interpretability (states, logits, halting).

Adaptive computation and halting. ACT allocates variable computation per input via a learned halting mechanism (Graves, 2016). In iterative solvers, halting provides an additional behavioral axis (steps taken) that can correlate with puzzle difficulty and reasoning depth.

Sparse autoencoders for interpretability. SAEs are used to decompose model activations into sparse features intended to be more interpretable than neurons in superposition (Elhage et al., 2022). Cunningham et al. show SAEs can recover highly interpretable features and support fine-grained causal tests (Cunningham et al., 2023). Gao et al. study scaling laws and evaluation metrics for large SAEs and advocate Top-K sparsity for stable training (Gao, 2025); Top-K variants also connect to k-sparse autoencoders (Makhzani and Frey, 2013) and classic sparse coding (Olshausen and Field, 1996).

Causal/mechanistic interpretability methods. Our interventions are in the spirit of causal testing of internal hypotheses: activation patching and its best practices (Heimersheim and Nanda, 2024), rigorous resampling ablations via causal scrubbing (Chan et al., 2022), and causal abstraction as a general framework (Geiger et al., 2023). Work on localized causal effects in transformers (e.g., ROME) further motivates measuring behavioral impact of targeted interventions (Meng et al., 2022). Recent safety work also connects SAE latents to high-level behaviors (OpenAI, 2025; Wang et al., 2025), suggesting SAEs can sometimes expose behaviorally-relevant directions.

3 Dataset

We conduct experiments on ARC-AGI-1 Chollet (2019), a benchmark of abstract visual reasoning tasks. We use the `arc1concept-aug-1000` dataset variant, which applies 1000 augmentations to each puzzle. For the ablation study, we select 20 representative, unique validation puzzles spanning a range of difficulty levels and task types.

Each ARC task consists of a small set of training input–output pairs and a set of test inputs:

$$\mathcal{T} = \{(X_i, Y_i)\}_{i=1}^{n} \ \cup \ \{(X_{\text{test}}^{(j)}, Y_{\text{test}}^{(j)})\}_{j=1}^{m},$$

where X and Y are discrete 2D color grids (Chollet, 2019), n is the number of training examples, and m is the number of test inputs for the task.

The model is provided with the training pairs $\{(X_i, Y_i)\}_{i=1}^{n}$ and all test inputs $\{X_{\text{test}}^{(j)}\}_{j=1}^{m}$, and must predict the corresponding outputs $\{Y_{\text{test}}^{(j)}\}_{j=1}^{m}$. A task is considered solved only if the model correctly predicts the output for every test input associated with that task.

4 Background

4.1 Tiny Recursive Model (TRM)

We use a compact ACTV1 TRM with two working memories: a lower-level stream $z_L^{(s)}$ and a higher-level stream $z_H^{(s)}$. Each token is $\mathbb{R}^{512}$ vector, and we prepend a 16-token puzzle-embedding prefix, so each stream has length $T + 16$. At each reasoning step, the TRM runs a small 2-layer transformer block:

1. z_L is updated using the input and z_H
2. z_H is updated using the new z_L.

In our config this happens for 6 lower-level updates and 3 higher-level updates per step. The LM head reads logits from z_H (excluding the prefix), and the halting head reads the first prefix token. ACT caps the number of steps at $S_{\max} = 16$; at evaluation we always run all 16 steps for batching and log z_L trajectories for interpretability.

4.2 Sparse AutoEncoder (SAE)

Sparse Autoencoders (SAEs) are a self-supervised learning technique that decomposes dense neural activations into a small set of sparse, reusable features. By enforcing sparsity, e.g., often via ℓ_1 regularization or Top-K constraints, SAEs aim to mitigate feature superposition and recover more interpretable directions in hidden states (Cunningham et al., 2023). Recent work shows that SAEs trained on internal activations of large models can yield features that align with semantically or

behaviorally meaningful concepts and support causal interventions (Cunningham et al., 2023; Gao, 2025). In this work, we use a Top-K SAE as a lightweight tool to probe and intervene on the latent reasoning trajectories of a recursive model.

5 Method

In this section, we illustrate how we trained SAE and use it for interpretation of TRM's latent reasoning trajectory.

5.1 SAE Training

Dataset. We use the `arc1concept-aug-1000` dataset variant which is same dataset that used for training TRM. We used SAE checkpoint with the one with minimal validation loss.

Architecture. We train a Sparse Autoencoder (SAE) on the TRM's internal hidden states z_L. The encoder projects each hidden state from $\mathbb{R}^{512}$ into a 4,096-dimensional feature space, followed by a ReLU activation and a Top-K sparsity constraint (Makhzani and Frey, 2013; **?**). The decoder then maps the sparse representation back to $\mathbb{R}^{512}$. The SAE additionally includes a learnable pre-encoder bias $b_{\text{pre}} \in \mathbb{R}^{512}$ and an encoder bias $b_{\text{enc}} \in \mathbb{R}^{4096}$.

Hyperparameters. Table 1 summarizes the SAE configuration. Using a dictionary of 4,096 features with Top-64 sparsity results in approximately 1.5% active features per token, providing a balance between representational capacity and interpretability (Gao, 2025).

Parameter	Value
Hidden dimension (d_{model})	512
Dictionary size (n_{features})	4,096
Top-K sparsity	64
Auxiliary loss weight (α)	$1/32$
Dead feature threshold	200,000 tokens

Table 1: SAE hyperparameters.

Encoding Process. Given hidden states z_L, the SAE computes sparse feature activations according to

$$z_n = \text{TopK}_{64}\left(\text{ReLU}\left(W_{\text{enc}}(z_L - b_{\text{pre}}) + b_{\text{enc}}\right)\right), \tag{1}$$

where $W_{\text{enc}} \in \mathbb{R}^{4096\times 512}$ denotes the encoder dictionary. For each token, only the 64 largest activations are retained.

Reconstruction. The decoder reconstructs the hidden state as

$$\hat{z}_L = W_{\text{dec}} \cdot z_n + b_{\text{pre}}, \tag{2}$$

where $W_{\text{dec}} \in \mathbb{R}^{512\times 4096}$ is the decoder dictionary.

5.2 Ablation Methodology

To study the causal role of SAE features in the model's reasoning process, we perform systematic ablation experiments consisting of feature ranking, targeted intervention, and evaluation.

Feature Ranking We compute per-feature scores over a selected set of positions P (steps/tokens, optionally answer-only) and sort features by each score.

$$\text{AvgAct}_i = \frac{1}{|P|}\sum_{(s,t)\in P} a_{s,t,i}, \qquad \text{Freq}_i = \sum_{(s,t)\in P} \mathbf{1}[a_{s,t,i} > 0]. \tag{3}$$

Intervention Methods Given a set of target features $\mathcal{F} = \{f_1, \ldots, f_K\}$, we apply interventions at every reasoning iteration. We consider two intervention strategies.

In the *Subtract* method, we compute the contribution of the target features to the hidden state and subtract it from the original representation:

$$z_n^{\text{masked}} = z_n \odot \mathbf{m}, \quad m_i = \mathbb{1}[i \in \mathcal{F}], \tag{4}$$

$$z_L^{\text{abl}} = z_L - W_{\text{dec}} \cdot z_n^{\text{masked}}. \tag{5}$$

This intervention removes the selected feature contributions while preserving information not captured by the SAE.

In the *Reconstruct* method, we zero out the target features and replace the hidden state with the SAE reconstruction:

$$z_n^{\text{remaining}} = z_n \odot (1 - \mathbf{m}), \tag{6}$$

$$z_L^{\text{abl}} = W_{\text{dec}} \cdot z_n^{\text{remaining}} + b_{\text{pre}}. \tag{7}$$

This intervention is more aggressive, as it discards any information not represented in the SAE.

Progressive Ablation Protocol Rather than ablating individual features, we perform progressive ablation in groups to reduce computational cost. Features are first ranked by importance, after which we evaluate ablation levels $K \in \{100, 200, 300, \ldots, 4000, 4096\}$. For each value of K, the top-K ranked features are ablated, the model is run with intervention applied at every reasoning iteration, and the resulting predictions and error metrics are recorded. This procedure yields 41 ablation checkpoints per puzzle and allows us to trace how performance degrades as increasingly important features are removed. We further analyze error change points, defined as values of K at which the number of prediction errors changes, to identify feature groups that are causally responsible for specific components of the solution.

6 Results

6.1 Qualitative analysis

We perform a qualitative analysis to understand how progressive ablation of SAE features affects the reasoning behavior of the TRM when operating on reconstructed trajectories. Figures 1 and 2 visualize model predictions, for two particular examples, under increasing ablation levels K, where the topK SAE features, ranked by importance, are removed using the *Reconstruct* intervention at every reasoning iteration.

Across multiple puzzles, progressive SAE ablation does not cause an immediate failure of the prediction. Instead, as K increases, the overall shape and direction of the solution trajectory are preserved, while the model exhibits structured and localized changes in its prediction, including incorrect turns, spatial offsets, or misaligned segments. This pattern indicates that TRM's recursive reasoning is partitioned across functional groups of SAE features. While the overall structure is supported by features that are redundant and broadly distributed, finer local decisions depend on a smaller set of more specific, high-importance features.

Notably, the number of prediction differences does not increase monotonically as more features are ablated. This non-monotonic pattern suggests that the SAE features do not act independently. Removing extra features can sometimes reduce interference, resulting in stable reconstructions and predictions. This points to redundancy and interaction in the SAE feature space, rather than a strict importance ordering.

We identify a class of puzzles for which ablating all SAE features from z_L produces no observable change in the model's prediction. This indicates that, for these tasks, the TRM relies primarily on the higher-level representation z_H to determine the output. In such cases, z_L may serve a transient or auxiliary role during computation, while the decisive reasoning occurs in z_H. This observation highlights a fundamental limitation of interpretability analyses that focus on a single latent stream and suggests that future work should jointly analyze and intervene on both z_L and z_H to obtain a complete causal picture.

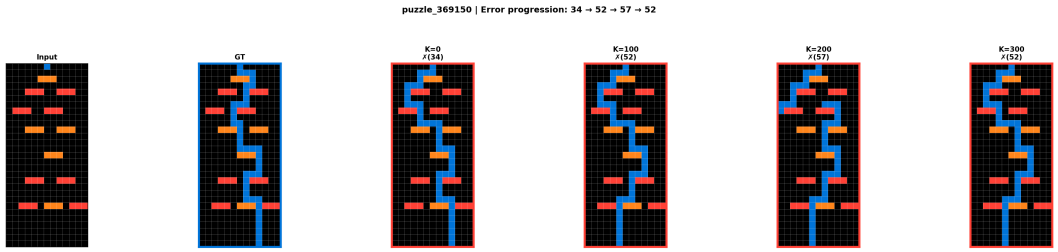

Figure 1: **Qualitative example 1** *Left*: input puzzle. *Second*: ground-truth output. Remaining columns show the model's predictions under progressive SAE feature ablation, with increasing ablation level K (number of ablated features).

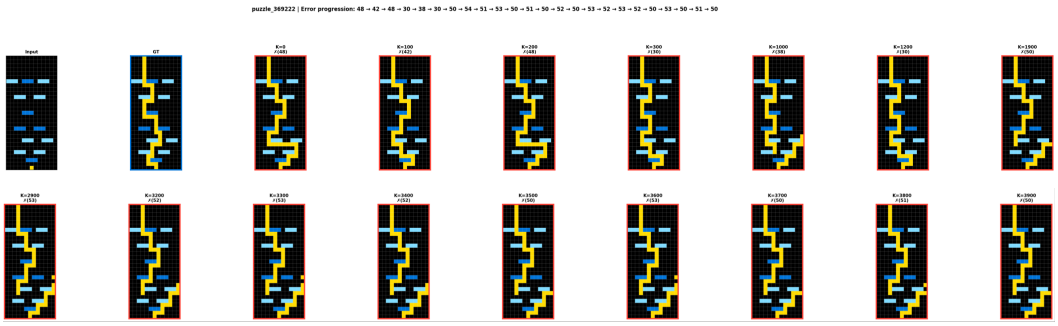

Figure 2: **Qualitative example 2** *Left*: input puzzle. *Second*: ground-truth output. Remaining columns show the model's predictions under progressive SAE feature ablation, with increasing ablation level K (number of ablated features).

6.2 Failure cases

Figures 3 and 4 illustrate two failure cases that highlight the difficulty of learning and interpreting continuous reasoning trajectories using the current SAE formulation.

In Figure 3, we present examples where ablating *all* SAE features results in no change in the final prediction. Despite removing the entire contribution captured by the SAE, the model's output remains identical to the unablated case. This suggests that, for a subset of training samples, the TRM does not rely on the intermediate hidden states z_L for solving the reasoning task. Instead, the solution appears to be determined primarily by the higher-level representation z_H. This observation indicates that interpreting z_L alone may be insufficient for fully characterizing the TRM's reasoning behavior, and that understanding the role of z_H is critical for comprehensive interpretability.

Figure 4 shows a different failure mode, in which progressive SAE ablation leads to non-monotonic corruption of the model's predictions. Rather than exhibiting a gradual degradation as more features are removed, the model's output can change and then revert back to a previous prediction. In the figure, two predictions highlighted by orange brackets demonstrate this behavior: as additional features are ablated, the output first changes to an incorrect solution and then returns to a previous prediction. Such non-monotonic behavior makes causal interpretation of individual SAE features particularly challenging.

We hypothesize that this phenomenon arises from two factors. First, interference from the higher-level representation z_H can partially compensate for the removal of certain SAE features. Second, our current sparsification strategy applies a Top-K constraint independently at the token level, analogous to common practices in language models. However, unlike language modeling—where each token introduces new information—ARC puzzles are structured such that most relevant information is already contained in the input grid. Applying token-wise Top-K sparsity may therefore introduce unnecessary variability and instability in the sparse representation. A more appropriate approach may be to impose sparsity at the puzzle level, selecting a shared set of Top-K features across all

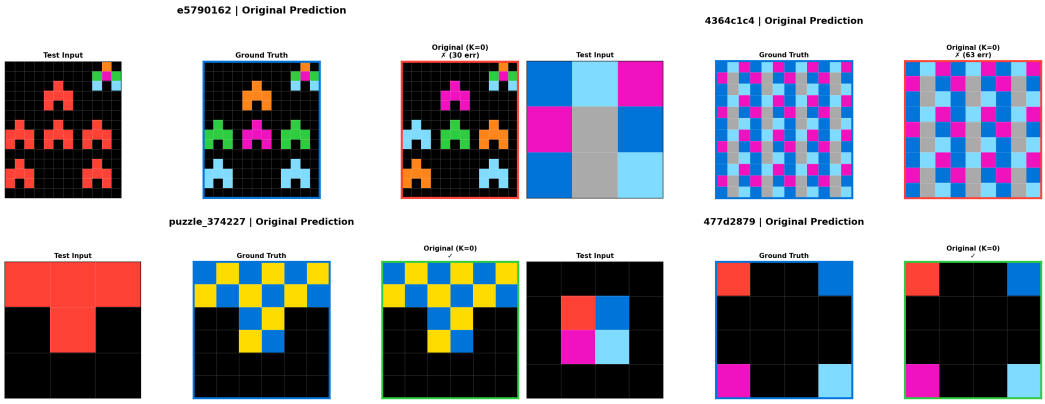

Figure 3: **Failuer case 1: SAE ablation with no effect.** Ablating all SAE features does not change the model's final prediction for certain puzzles, indicating that the solution does not rely on the intermediate hidden states z_L.

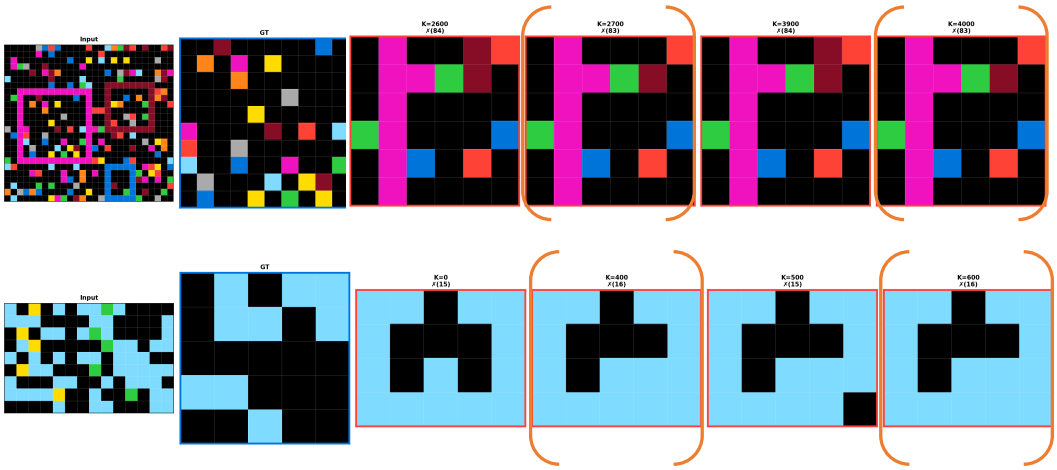

Figure 4: **Failuer case 2: Non-monotonic behavior under SAE ablation.** Progressive feature ablation leads to prediction changes that are not monotonic, with outputs changing and reverting as more features are removed.

tokens within a puzzle. We believe that such a puzzle-level sparsity constraint could lead to more stable and interpretable SAE features, and we leave this direction for future work.

7 Conclusion

We proposed a feature-level interpretability pipeline for a Tiny Recursive Model (TRM) on ARC-AGI-1, combining trajectory logging, Top-K sparse autoencoders, and causal interventions during recurrence. This framework enables systematic probing of how internal latent features influence puzzle-solving behavior beyond final accuracy. Our results show that SAE-based ablations can reveal structured causal effects, but also expose clear limitations: some solutions are unaffected by removing all SAE features, and progressive ablation can lead to non-monotonic behavior. These findings highlight both the utility and the challenges of applying sparse feature decompositions to recurrent and adaptive-computation models.

References

Lawrence Chan, Adrià Garriga-Alonso, Nicholas Goldwosky-Dill, Ryan Greenblatt, Jenny Nitishinskaya, Ansh Radhakrishnan, Buck Shlegeris, and Nate Thomas. Causal scrubbing: a method for rigorously testing interpretability hypotheses. *AI Alignment Forum*, 2022.

François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. *arXiv preprint arXiv:2209.10652*, 2022.

Atticus Geiger, Christopher Potts, and Thomas Icard. Causal abstraction for faithful model interpretation. *arXiv preprint arXiv:2301.04709*, 2023.

Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255*, 2024.

Gao, Leo and la Tour, Tom Dupré and Tillman, Henk and Goh, Gabriel and Troll, Rajan and Radford, Alec and Sutskever, Ilya and Leike, Jan and Wu, Jeffrey. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.

Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.

Alireza Makhzani and Brendan Frey. k-sparse autoencoders. *arXiv preprint arXiv:1312.5663*, 2013.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. *Advances in Neural Information Processing Systems*, 2022.

Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381:607–609, 1996.

OpenAI. Toward understanding and preventing misalignment. OpenAI blog, June 18, 2025.

Miles Wang, Tom Dupré la Tour, Olivia Watkins, Alex Makelov, Ryan A. Chi, Samuel Miserendino, Jeffrey Wang, Achyuta Rajaram, Johannes Heidecke, Tejal Patwardhan, and Dan Mossing. Persona features control emergent misalignment. *arXiv preprint arXiv:2506.19823*, 2025.

Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.